Expert Systems With Applications 300 (2026) 130219

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Review

Autonomous penetration testing using reinforcement learning: A review and perspectives

Jingju Liu [a,b,c,1], Yue Zhang [d,1], Shicheng Zhou [b,c,*], Jiahai Yang [a,*], Yuliang Lu [b,c], Xiaofeng Zhong [b,c]

[a] Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, 10084, China
[b] College of Electronic Engineering, National University of Defense Technology, Hefei, 230037, China
[c] Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, Hefei, 230037, China
[d] College of Computer Science and Technology, National University of Defense Technology, Changsha, 410073, China

ARTICLE INFO

Keywords:
Penetration testing
Reinforcement learning
Cybersecurity
Systematic review
Attack path.

ABSTRACT

Penetration testing (pentesting) assesses cybersecurity through controlled, authorized attacks, but traditional manual methods demand considerable human and time resources. Reinforcement learning (RL), with its agent-environment interaction paradigm, offers a promising approach for autonomous pentesting. Despite remarkable advancements in this field, there is a lack of comprehensive reviews and perspectives on RL-based autonomous pentesting. To address this gap, this paper presents a systematic review of RL-based autonomous pentesting research. We outline the key challenges faced when applying RL in autonomous pentesting and categorize the existing literature into two main areas: attack path planning and autonomous pentesting frameworks, based on the research objectives and hypotheses. Additionally, we offer an in-depth analysis of the latest advancements and limitations in this field, while proposing a perspective on future research directions in the field of RL-based autonomous pentesting. We hope that our work will provide valuable insights for researchers, contributing to the advancement of autonomous pentesting and its practical application in the complex and diverse scenarios of the real world.

1. Introduction

Penetration testing (shortly pentesting or PT) is an active and offensive cybersecurity evaluation methodology conducted under authorized conditions. Its goal is to analyze the attack surfaces and pinpoint potential vulnerabilities within target computer systems via controlled cyber attacks. Compared to traditional passive security protection methods, such as intrusion detection technologies, pentesting actively identifies potential threats from an attacker's perspective, providing a proactive approach to preventing security risks.

The traditional pentesting process is typically performed manually, relying on experts with extensive domain knowledge to gather information about the target systems and then select appropriate tools to verify potential vulnerabilities. However, as computer devices and networks have become more widespread, modern network systems have grown increasingly large and complex, with security vulnerabilities exposed on the internet becoming more frequent and diverse. This has resulted in an expanding attack surface, which means that traditional manual-based pentesting requires substantial human and time resources (Holm, 2023). In light of this, autonomous pentesting emerges as a promising solution.

Autonomous pentesting aims to develop an artificial intelligence (AI) agent that can autonomously analyze attack surfaces of target systems and identify potential vulnerabilities. Compared to the manual-based method, it offers advantages in terms of autonomy and efficiency (Zhou et al., 2025a).

Reinforcement learning (RL) (Sutton & Barto, 2018) is a natural fit for studying autonomous pentesting Zhou et al. (2025b). This suitability stems from two key factors. First, similar to domains like autonomous driving (Feng et al., 2023), robotics (Zhao et al., 2020), and competitive gaming (Vinyals et al., 2019), pentesting can also be viewed as a sequential decision-making process based on environmental feedback. During the pentesting process, testers dynamically adjust their policies in response to target feedback (e.g., scan results) to progressively identify and exploit weaknesses. This process inherently reflects the testers interacting with an environment (target system), optimizing decisions

* Corresponding authors.
E-mail addresses: liujingju17@nudt.edu.cn (J. Liu), zhangyue@nudt.edu.cn (Y. Zhang), zhoushicheng@nudt.edu.cn (S. Zhou), yang@cernet.edu.cn (J. Yang), luyuliang@nudt.edu.cn (Y. Lu), zhongxiaofeng17@nudt.edu.cn (X. Zhong).
[1] The two authors contribute equally to this work.

https://doi.org/10.1016/j.eswa.2025.130219
Received 24 July 2025; Received in revised form 30 September 2025; Accepted 27 October 2025
Available online 10 November 2025

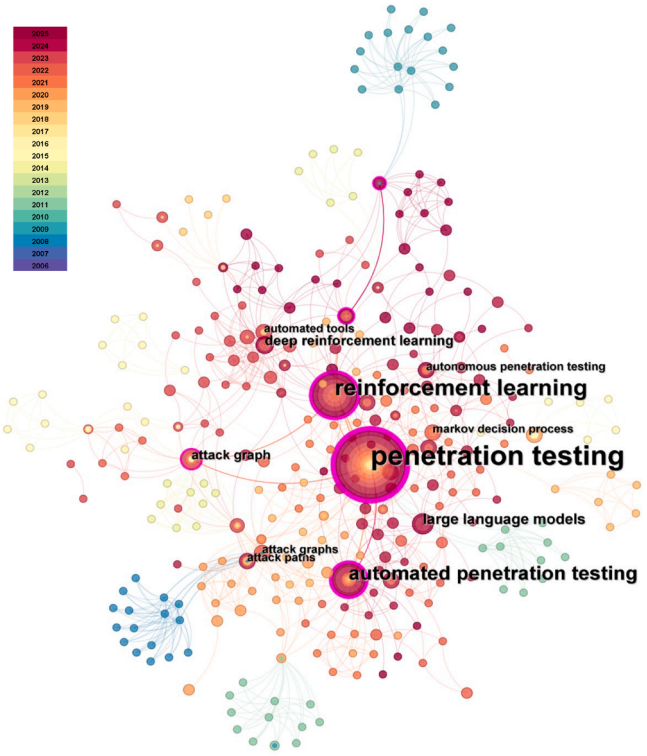

Fig. 1. The keyword co-occurrence network in the autonomous pentesting field.

through trial-and-error to achieve goals. Second, in machine learning (ML), RL is particularly well-suited to sequential decision-making problems (Sutton & Barto, 2018). Unlike supervised or unsupervised learning, RL provides a learning paradigm of training agents to learn policies through trial-and-error while interacting with the environment, requiring neither pre-labeled data nor environment models. This makes it capable of handling the dynamic, exploratory nature of pentesting.

Autonomous pentesting is a challenging research goal that has undergone a long development process. Fig. 1 presents a keyword co-occurrence network, showcasing the research hotspots in the field of autonomous pentesting based on works indexed in the Web of Science (WoS) database [1] from 2006 to 2025. The co-occurrence map highlights the evolution of key research themes and areas of focus in this domain over nearly two decades.

Autonomous pentesting originated from attack planning (Hoffmann, 2015; Sarraute, 2013). Early approaches primarily relied on formal analysis methods, including Planning Domain Definition Language (PDDL) (Obes et al., 2013), classic network threat modeling frameworks (such as attack graphs and attack trees (Phillips & Swiler, 1998; Schneier, 1999; Sheyner et al., 2002)), and certain probabilistic modeling methods (such as Partially Observable Markov Decision Processes (Sarraute et al., 2013; Schwartz et al., 2020)). These methods depend on explicitly constructing system models in advance (e.g., graphs, trees, or probabilistic transition models) along with comprehensive environmental prior knowledge. The advantages lie in their intuitiveness and strong interpretability, but due to high computational complexity, they are inefficient when conducting attack planning in large-scale network environments.

Recently, with the significant advancements made in RL and large language models (LLMs) in the field of intelligent decision-making, both RL-based and LLM-based methods have indeed become promising solutions. Compared to RL-based methods, LLM-based methods (Deng et al., 2024) offer advantages in their superior contextual semantic understanding and the ability to produce diverse decision outcomes. Besides, LLMs can inject domain-specific knowledge via post-training, resulting in better adaptability to specific tasks (Kobayashi et al., 2025). However, as generative language models, LLMs are still constrained by their inherent characteristics, facing challenges such as hallucination when relying entirely on LLMs for intelligent decision-making, which may lead to inaccurate or unstable decision outputs (Moskal et al., 2023; Wu et al., 2024). LLM-based methods are a promising area for future exploration, while they fall beyond the scope of this work. For further details, readers are referred to Happe and Cito (2025), Isozaki et al. (2025).

Scope and Motivation of this Review. With the rapidly growing interest in autonomous pentesting research, there have been extensive reviews of a large body of work in this topic, such as Abu-Dabaseh and Alshammari (2018), Rane and Qureshi (2024), Saber et al. (2023), Skandylas and Asplund (2025). Previous reviews have primarily focused on the standards, processes, and automation tools used across various stages of pentesting, particularly emphasizing comparisons between manual and automated approaches. However, there has been a lack of comprehensive and systematic reviews on the decision-making methods within autonomous pentesting, especially those based on RL. To address this gap, this paper mainly discusses the literature on the application of RL-based decision-making methods in autonomous pentesting, including algorithm design for path planning and automation tools, key challenges, and existing limitations.

Literature Search and Selection. We performed a systematic literature search across multiple databases, including Google Scholar, Web of Science , and Scopus. The search strategy involved keywords such as “reinforcement learning,” “penetration testing,” and “attack planning,” combined using Boolean operators (AND/OR). The search was conducted in English-language publications only. In addition to cybersecurity, we also explored research on RL in fields such as autonomous driving and robotics, as they share similar challenges related to decision-making and optimization. Relevant studies from these fields were reviewed to provide a broader perspective on the application of RL in complex, real-time decision-making processes.

Terminology Note. While both “automated penetration testing” (Skandylas & Asplund, 2025) and “autonomous penetration testing” (Zhou et al., 2025a) appear in existing literature, this paper adopts the latter terminology to precisely reflect our research focus. The term autonomous (versus automated) intentionally emphasizes the agent’s capacity for adaptive decision-making in various environments (like autonomous vehicles (Feng et al., 2023; Xu et al., 2018) and autonomous driving (Wenzel et al., 2025)). Whereas automated systems typically execute predefined procedures, autonomous agents, particularly those RL-based agents, continuously evaluate environmental feedback to self-optimize attack strategies through exploration and exploitation. We contend that “autonomous” more accurately captures the intelligent, self-directed nature of RL-driven pentesting explored in this work.

Key Contributions. The main contributions of this paper are threefold: (1) a systematic review of RL-based autonomous pentesting research, categorized according to the research objectives and hypotheses of existing studies; (2) a summary and classification of the key challenges addressed by current works, along with an analysis of the limitations of these studies; and (3) perspectives into future research directions in the field of RL-based autonomous pentesting.

Overview of this Work. The remainder of this paper is organized as follows. Section 2 introduces the preliminaries of RL and pentesting. Section 3 provides a problem description, including the MDP (Markov Decision Process) model and OODA (Observe, Orient, Decide, Act) model for the pentesting process, as well as the key research challenges faced when applying RL algorithms to autonomous pentesting. We also categorize existing literature according to the research objectives and hypotheses in this section. Section 4 discusses RL-based attack path planning methods. Section 5 presents RL-based pentesting frameworks. Then, Section 6 offers future perspectives on RL-based autonomous pentesting. Finally, Section 7 concludes this review.

[1] https://clarivate.com.cn/solutions/web-of-science/

2. Preliminary

2.1. Reinforcement learning

RL trains agents to learn optimal policies that maximize cumulative rewards by interacting with an environment. This agent-environment interaction paradigm inherently aligns with autonomous pentesting, making RL a natural fit for studying autonomous pentesting.

Typically, an RL problem can be modeled as a finite, discrete-time MDP that is formalized by a tuple $(S, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$, where S represents the state space, $\mathcal{A}$ is the action space, $\mathcal{R} : S \times \mathcal{A} \mapsto \mathbb{R}$ is the reward function, $\mathcal{P} : S \times \mathcal{A} \times S \mapsto [0, 1]$ is the state transition probability function and γ is the discount factor used to determine the importance of long-term rewards (François-Lavet et al., 2018; Sutton & Barto, 2018).

The concepts of agent and environment are crucial in the context of RL. The agent is the entity that undergoes learning, while the environment refers to the entity or virtual space with which the agent interacts. Additionally, the term *scenario* is also commonly used in RL, which typically refers to a specific configuration of the environment within the context of a particular task.

Formally, for a standard RL framework, at each time step t , the RL agent observes the state s_t from the environment and selects an action a_t based on its policy π. Then, the agent receives a reward signal from the environment, and the environment transitions to the next state s_{t+1} . The goal of the RL agent is to find the optimal policy π^* with internal parameters θ^* that maximizes the expected cumulative reward (Wang et al., 2023). The objective function of RL algorithms can be defined as

$$J(\theta) = \mathbb{E}_{\tau \sim D_{\pi_\theta}} \sum_{t=0}^{T} \gamma^t R_t, \tag{1}$$

where $\gamma \in [0, 1]$ is the discount factor used to determine the importance of long-term rewards, τ is a trajectory $(s_0, a_0, s_1, a_1, \ldots, s_T)$, D_{π_θ} denotes the distribution of τ under the policy π_θ.

The integration of deep learning with RL has driven the emergence of Deep Reinforcement Learning (DRL) (Arulkumaran et al., 2017). Compared to traditional RL algorithms (e.g., tabular Q-learning (Melo, 2001)), DRL algorithms (e.g., Deep Q-learning (Mnih et al., 2015)) leverage deep neural networks to process high-dimensional states and action spaces, making them particularly effective for complex decision-making tasks. As DRL constitutes an advanced subset of RL and represents the current mainstream methodology, this paper treats “reinforcement learning (RL)” as an umbrella term encompassing both classical RL and DRL approaches, thereby eliminating unnecessary terminological fragmentation.

2.2. Penetration testing

Pentesting is usually used to assess network security by implementing possible hacking cyber attacks Sarraute et al. (2012). It involves executing controlled attacks on the target systems or a network in order to find potential vulnerabilities.

Pentesting methodologies are categorized as white-box, gray-box, or black-box, based on the level of internal knowledge granted to the tester about the target system (Midian, 2002). Both white-box and gray-box testing provide the tester with access to internal system details such as network topology, host configurations, or login credentials. The distinction lies in information comprehensiveness: white-box testing grants full access, while gray-box testing offers only partial information. Although valuable for identifying vulnerabilities leveraging privileged knowledge, these approaches inherently struggle to model the perspective of an external attacker. Conversely, black-box testing grants the tester no prior internal knowledge of the target system. This approach aims to model the perspective of a genuine external attacker operating without privileged insight, thus providing a more realistic simulation of real-world attack scenarios.

In the context of pentesting, the Penetration Testing Execution Standard (PTES) [2] provides a standardized process developed by cybersecurity experts. Similar frameworks include the ATT&CK model [3] and the Kill Chain model (Yadav & Rao, 2015), all offering structured methodologies for understanding and mitigating cyber threats. While each framework has a unique focus, they share common objectives.

A generalized pentesting process can be simply conceptualized through the following core phases: Information Gathering, Vulnerability Exploitation, and Post Exploitation activities (Zhou et al., 2019). Information Gathering involves actively probing the target system to identify assets and potential attack surfaces. This includes discovering open ports, running services, operating systems (OS), application types (e.g., web server fingerprint), etc. This data fuels subsequent vulnerability analysis. Vulnerability Exploitation leverages identified flaws to gain unauthorized access or compromise system integrity. Ethical penetration testers conduct exploitation solely to validate vulnerability existence and impact, providing critical evidence for remediation reports. Tools like exploit code (EXP) or payloads are utilized for this verification. Post-Exploitation phases analyze the compromised environment, seek privilege escalation, identify data exposure, and evaluate potential lateral movement - all crucial for understanding breach impact. The entire process inherently involves sequential attacker actions, making it well-suited for modeling as a sequential decision-making problem.

3. Problem description

3.1. Model penetration testing as an RL problem

RL fundamentally relies on the Markov Decision Process (MDP) as its mathematical framework. Therefore, by modeling the pentesting process as an MDP, RL algorithms can be employed to train the pentesting agent.

In the context of MDP, **the state space** S of a pentesting agent consists of all observable cybersecurity features that provide the basis for the agent’s decision-making process. The state information is extracted from environmental feedback that primarily includes host-related details associated with vulnerabilities, such as ports, services, and operating systems, which assist the agent in inferring potential vulnerabilities. In some works, for the sake of idealized modeling, the state space may also encompass network-related features (e.g., network topology, number of hosts) and defense mechanisms, despite the fact that such information is often difficult to observe directly. **The action space** $\mathcal{A}$ consists of penetration operations, such as scanning and vulnerability exploitation. The agent can interact with the training environment either through the actual use of relevant pentesting tools or by logical interaction, thereby receiving feedback and reward signals from the environment. The definition of the **reward function** $\mathcal{R}$ relies on subjective estimates from pentesting experts and varies depending on the training objectives. In most studies, the reward function is typically defined as a balance between the attack benefits (positive reward values) and the costs associated with pentesting actions (negative reward values). However, assessing reward function presents a challenge since there’s no objective standard defining an “optimal” strategy (Holm, 2023).

As depicted in Fig. 2 , the agent-environment interaction can also be viewed as an OODA loop (Brehmer, 2005), providing a complementary perspective to the MDP perspective (Holm, 2023; Zhou et al., 2025b): (a) **Observe**: the agent perceives environmental feedback (e.g., scan responses, exploit outcomes). (b) **Orient**: raw state information extracted from environmental feedback is then encoded into a structured state vector consumable by decision-making neural networks. (c) **Decide**: an optimal action is selected according to the current policy. (d) **Act**: the

[2] http://www.pentest-standard.org/index.php/Main_Page
[3] https://attack.mitre.org/

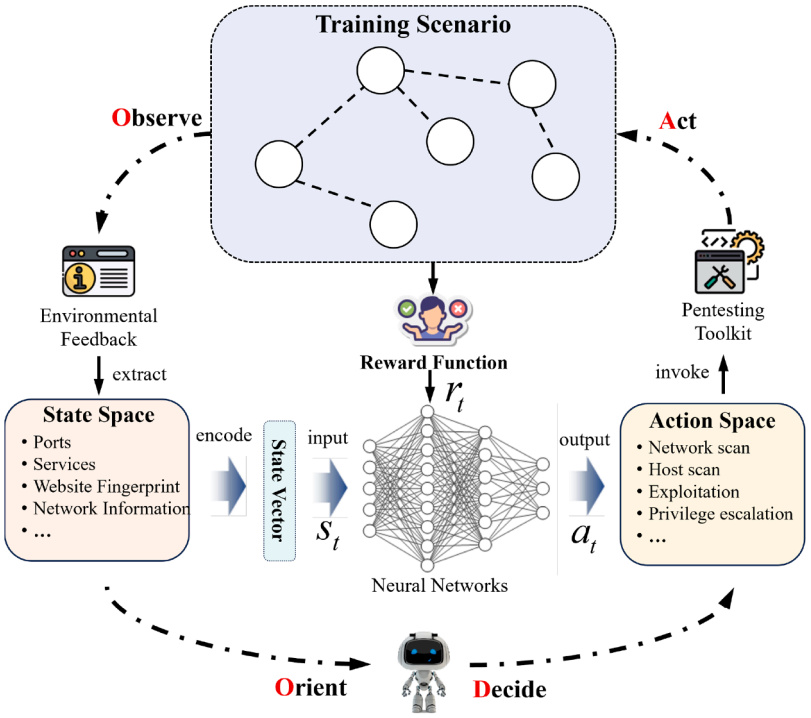

Fig. 2. RL agent design for autonomous penetration testing: MDP and OODA perspectives.

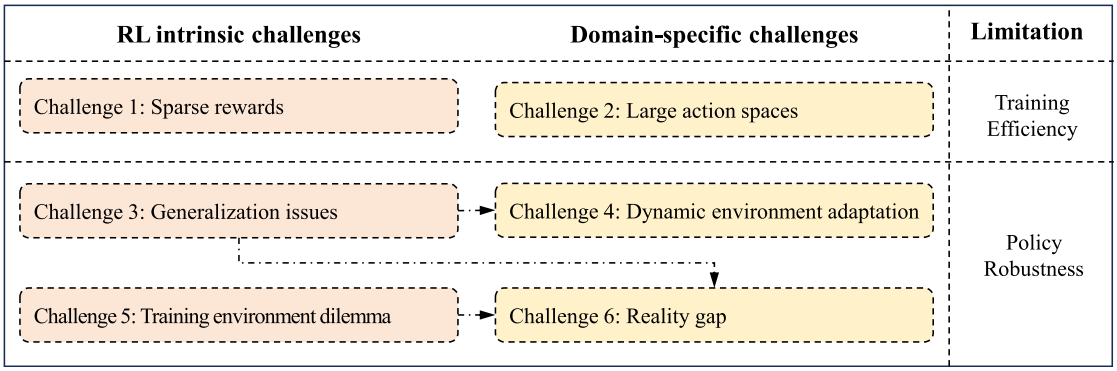

Fig. 3. Key challenges in RL-based autonomous penetration testing.

agent executes the action by invoking pentesting tools to alter the environment.

Note that some research (e.g., Ren et al. (2024a,b), Zhang et al. (2022)) also models the pentesting process as Partially Observable Markov Decision Processes (POMDPs), thereby modeling the uncertainty in the agent's pentesting training process. They assume that the environmental state information is partially observable to the pentesting agent. By introducing belief states, the agent maintains and updates the probabilistic estimation of the hidden network state and seeks to make optimal decisions in the sense of maximizing long-term expected rewards.

3.2. Key challenges

When training pentesting agents using RL algorithms, several key challenges typically arise. Some of these challenges are domain-specific, while others are inherent limitations of RL algorithms.

Challenge 1: Sparse rewards.

Sparse rewards (Weiyi et al., 2020) refer to a situation in RL where the agent receives feedback infrequently or only upon achieving critical milestones (e.g., successfully compromising a target host in pentesting tasks), making it difficult for the agent to understand the relationship between its actions and the eventual reward (Silver et al., 2021). This contrasts with dense-reward scenarios where incremental progress receives frequent feedback (Pathak et al., 2017). Sparse rewards may slow down the learning process, increase the number of training episodes required, and even lead to poor performance or suboptimal behavior as the agent fails to explore the environment sufficiently.

Challenge 2: Large action spaces.

In RL, agents optimize their policies by balancing exploration (trying new actions to discover rewards) and exploitation (using the current learned policy to select actions). Large action spaces severely hinder this process: as the agent must evaluate exponentially more possibilities to find optimal policies, exploration becomes inefficient (Dulac-Arnold et al., 2015; Fourati et al., 2024). This challenge is inherent to pentesting agents, where growing tool diversity and increasing volumes of publicly disclosed vulnerabilities constantly expand the action space (Zhou et al., 2025a).

Challenge 3: Generalization issues.

RL algorithms tend to overfit the training environments (Cobbe et al., 2019). As a result, agents' learned policies may perform poorly when applied to unseen testing (real-world) scenarios (Wang et al., 2020).

The issue is further compounded by the fact that even a minor change in the environment may necessitate retraining the agent's policy. This challenge is also referred to as the generalization gap (Kirk et al., 2023).

Challenge 4: Dynamic environment adaptation.

Pentesting scenarios in real-world settings are non-stationary (Zhou et al., 2025b). This non-stationarity arises because the attack surfaces are constantly evolving (e.g., services may restart with new configurations, and vulnerabilities are patched). As a result, agents are expected to adapt their policies to remain effective in dynamic environments.

Challenge 5: Training environment dilemma.

Similar to real-world applications like robotics (Ju et al., 2022) and autonomous driving (Xu et al., 2018), directly training pentesting agents in real-world scenarios is often impractical. This is due to two main reasons : (a) agents' trial-and-error exploration risks causing unpredictable damage to real-world systems through destructive actions (e.g., vulnerability exploitations); (b) the extended time between action execution and environmental feedback creates critical bottlenecks for experience sampling efficiency (Ju et al., 2022). To address this, agents are often trained in simulated or emulated environments, though ensuring these environments accurately reflect the real world remains a challenge (Zhou et al., 2024).

Challenge 6: Reality gap.

The reality gap (Tobin et al., 2017) describes the mismatch between limited-diversity training environments and the unpredictable diversity of unseen testing (real-world) scenarios. Combined with the overfitting issues in RL algorithms, agents trained in simulations often struggle when deployed in real-world situations.

Fig. 3 illustrates the key challenges mentioned above. Among these challenges, Challenge 1 and Challenge 2 primarily limit the training efficiency by affecting the agent's exploration efficiency. Conversely, Challenge 3 to Challenge 6 primarily impact agents' policy robustness, which manifests as the policy's failure to maintain performance when transferred to unseen/dynamic testing (real-world) scenarios.

There also exist certain interconnections between Challenge 3 and Challenge 6. The overfitting of RL algorithms to the training environment results in poor policy generalization (Challenge 3), which further leads to poor performance when the agent faces dynamic environments (Challenge 4). At the same time, the existence of the reality gap (Challenge 6) makes it difficult for the agent to cope with real-world environments that differ from the training environment. Additionally, as depicted in Fig. 4, the environmental dilemma (Challenge 5) requires a balance between sampling efficiency and environmental fidelity in the training environment, which inevitably contributes to the existence of the reality gap (Challenge 6).

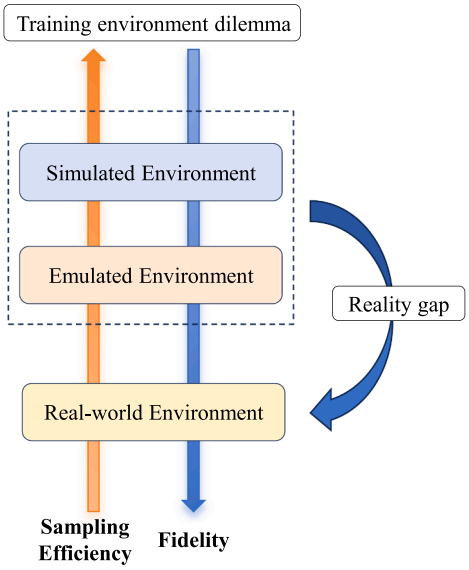

Fig. 4. Training environment dilemma and reality gap.

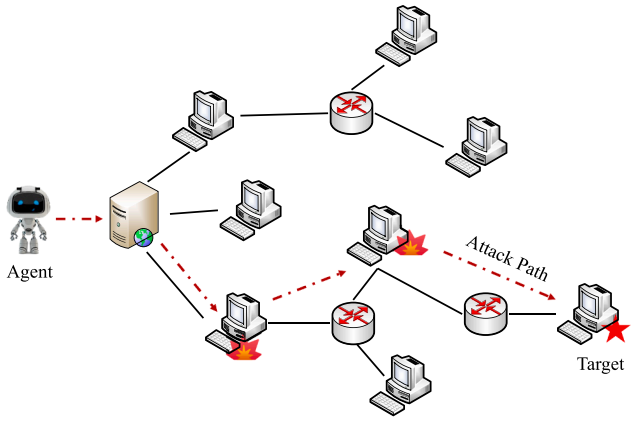

Fig. 5. Attack path.

3.3. Taxonomy

Autonomous pentesting can effectively reduce time and labor costs, providing a continuous, regular assessment of cybersecurity posture from an attacker's perspective. Autonomous pentesting is a highly practical research area where researchers often need to first define their specific research objectives before determining the scope of their work. From this perspective, we classify the existing research into **attack path planning** and **autonomous pentesting frameworks**, based on the core objectives and hypotheses underlying these studies. This objective-oriented taxonomy focuses on research objectives rather than on the specific methods or techniques used, which is the primary basis for taxonomy in earlier reviews Abu-Dabaseh and Alshammari (2018), Rane and Qureshi (2024), Saber et al. (2023), Skandylas and Asplund (2025). Table 1 presents the differences between the two categories of research.

Attack path planning refers to the process of modeling and planning possible attack paths based on known vulnerabilities, network topology, and attack vectors, with the aim of strategizing attack paths to identify the best path for an attacker to achieve specific objectives. It treats the autonomous pentesting problem as a planning problem (Hoffmann, 2015; Li et al., 2024; Sarraute, 2013), involving modeling potential attack routes, evaluating vulnerabilities, and selecting the best attack sequence.

Research on this topic typically hypothesizes that complete or partial environmental information is available, and based on this information, they construct training scenarios to train agents to plan the optimal pentesting path in specific simulated network scenarios. The pentesting path, also known as the attack path (see Fig. 5), represents the sequence of compromised hosts and associated actions taken to gain control of a target system. It originates from the initial reconnaissance phase and terminates upon achieving privileged access to the target host. An attack path P can be formally described as $P = \{(h_1, a_1), (h_2, a_2), \ldots, (h_t, a_t)\}$, where h_i, a_i represent the compromised host and the actions taken on the host, respectively. In order to improve the efficiency of the agent's interaction with the environment and ensure that the training process is safe and controllable, this type of research typically relies on using network simulators (such as NASim (Schwartz & Kurniawatti, 2019) and CyberBattleSim (Seifert et al., 2021)) to construct suitable simulated training scenarios. Therefore, network simulators and algorithm design are crucial in this field. A detailed introduction to this category of research is presented in Section 4.

Autonomous pentesting frameworks, on the other hand, refer to systems or frameworks that can conduct pentesting activities

Table 1
Comparison of attack path planning vs. autonomous pentesting frameworks.

Aspect	Attack path planning	Autonomous pentesting frameworks
Definition	Modeling and planning possible attack paths based on known vulnerabilities, network topology, and attack vectors.	Systems or frameworks that can autonomously conduct pentesting activities.
Objective	Strategizing attack paths to find the best path for an attacker to achieve specific goals.	Automating the entire pentesting process (or specific stage) by integrating real-world tools and AI techniques to mimic the behavior of human attackers.
Environment	Often network simulators.	Real vulnerable machines (typically virtual machines).
Hypothesis	Complete or partial environmental information is available.	Environmental information is unknown ; target machines are accessible.

Table 2
Representative works on RL-based autonomous penetration testing.

Challenges	Solution	Representative Work	Environment		Category
			Simulator	**Virtual Machine**	
Sparse rewards	improved experience replay	Li et al. (2025), Zhou et al. (2021)	✓		Attack Path Planning
	reward shaping	Zhou et al. (2019)		✓	Attack Path Planning
	incorporating expert knowledge	Chen et al. (2023)	✓		Attack Path Planning
	curiosity-driven intrinsic reward mechanisms	Chen et al. (2025), Yang et al. (2025), Zhang et al. (2022)	✓		Attack Path Planning
Large action spaces	hierarchical action space architectures	Li et al. (2023b), Tran et al. (2021);	✓		Attack Path Planning
	action mask	Li et al. (2023a), Yang et al. (2025)	✓		Attack Path Planning
	action embedding	Zhou et al. (2025a)		✓	Autonomous Pentesting Framework
	hierarchical architectures + action embedding	Nguyen and Uehara (2022)	✓		Attack Path Planning
Dynamic environment adaptation	structural inductive bias	Yang et al. (2023)	✓		Attack Path Planning
	historical trajectory logging	Li et al. (2024)	✓		Attack Path Planning
Training environment dilemma	sim-to-real transfer	Zhou et al. (2024)		✓	Autonomous Pentesting Framework
Reality gap	domain random	Zhou et al. (2024)		✓	Autonomous Pentesting Framework
Generalization issues	continual learning	Zhou et al. (2025b)		✓	Autonomous Pentesting Framework
	meta learning	Zhou et al. (2024)		✓	Autonomous Pentesting Framework

autonomously. These frameworks (e.g., Maeda and Mimura (2021), Takaesu (2018), Zhou et al. (2025a)) aim to automate the entire pentesting process (or specific stages, e.g., post-exploitation) by integrating real-world tools and AI techniques to mimic the behavior of human attackers, allowing for more practical and realistic pentesting. Unlike attack path planning, the focus here is on automating the pentesting process and identifying vulnerabilities without any prior knowledge of target environmental information.

Research on this topic does not rely on simulators to construct training environments; instead, it directly targets vulnerable machines, which are typically built using virtual machines. These studies usually hypothesize that the agent can directly facilitate actual interactions with the target machine and aim to design a suitable pentesting framework that autonomously invokes appropriate tools to compromise the target. The research focus and core challenges of this category lie in how to design the RL decision algorithms. We provide a detailed introduction to this type of research in Section 5.

The representative works of the two categories, along with their corresponding research challenges, proposed solutions, and experimental environments, are summarized in Table 2.

4. RL For attack path planning

4.1. Network simulator

RL-based attack path planning views the autonomous pentesting problem as a planning problem (Hoffmann, 2015; Li et al., 2024; Sarraute, 2013), aiming to train agents to plan the optimal pentesting path in specific simulated network scenarios. Limited by the training environment dilemma, previous research predominantly utilizes open-source benchmark environments, commonly referred to as network simulators.

NASim (Schwartz & Kurniawatti, 2019) and CyberBattleSim (Seifert et al., 2021) are widely used and representative network simulators for constructing digital training scenarios. Both focus on pentesting path planning and serve as open-source benchmark platforms for such research. These simulators employ declarative configurations (YAML[4] in NASim, Python scripts in CyberBattleSim) to define static network scenarios encompassing topology, firewall rules, host valuations, services, operating systems, and vulnerabilities. Within these abstracted

[4] https://yaml.org/

environments, agents seek optimal attack paths through logical scanning and exploitation actions, where optimality balances target value against operational costs.

However, both NASim and CyberBattleSim provide highly abstracted representations of network scenarios, which creates a significant gap between the training environment and real-world settings (reality gap). As a result, the path planning policy learned by the agent in these environments cannot be directly applied to real-world situations. To this end, PenGym (Nguyen et al., 2025) enhances NASim by leveraging KVM (Kernel-based Virtual Machine) technology, enabling agents to execute actual tools against emulated targets and receive actual environmental feedback. While PenGym improves agents' policy applicability to some extent, it is currently limited to specific NASim scenarios.

Beyond these environments, CybORG (Baillie et al., 2020) is another representative platform that supports red-blue team training. It combines NASim-style configuration with Amazon Web Services (AWS) command-line interfaces to configure emulated environments, allowing initial simulator training followed by emulator deployment. However, the complexity of building the emulated environment in CybORG is high, and the project has not been fully open-sourced. It is also reported that the development of the emulation module has already been discontinued (Simon & Mees, 2024).

4.2. Algorithm design

Some previous works (e.g., Hu et al. (2020), Yousefi et al. (2018)) on RL-based attack path planning combined RL with attack graphs to overcome the scalability limitations of traditional attack graph-based methods in large-scale networks. However, most of the existing work in this area has focused on improving the learning efficiency of agents or enhancing the robustness of their policies in dynamic simulated environments. These studies addressed the following key challenges when designing RL algorithms: (a) sparse rewards, (b) large action spaces, and (c) dynamic environment adaptation.

To address the problem of sparse rewards, several approaches have been proposed, including reward shaping techniques, intrinsic reward mechanisms, and improved experience replay methods. For instance, Zhou et al. (2021) and Li et al. (2025) enhanced the efficiency of utilizing historical experience data by introducing priority experience replay and hindsight experience replay mechanisms, respectively. Zhou et al. (2019), on the other hand, applied reward shaping by incorporating external information gain to design a dense reward mechanism that effectively guides the agent's policy learning. Additionally, Chen et al. (2025), Yang et al. (2025), Zhang et al. (2022) introduced curiosity-driven intrinsic reward mechanisms to boost the agent's exploration capabilities. These approaches collectively aimed to improve learning efficiency and promote more effective exploration in environments with sparse feedback.

Large action spaces pose challenges in RL algorithms due to exponential growth in computational complexity, sample inefficiency, and ineffective exploration (Chen et al., 2019; Dulac-Arnold et al., 2015). Solutions to address this problem include hierarchical action space architectures, action masking, and the incorporation of expert demonstration data. These methods mitigate the challenges by fundamentally restructuring or constraining the action space, allowing for more efficient learning and better handling of large action spaces. For example, Li et al. (2023b), Tran et al. (2021) proposed a hierarchical RL framework to decompose complex, high-dimensional actions into sequences of manageable lower-level sub-actions, thereby reducing the branching factor at each decision point. Nguyen and Uehara (2022) proposed a hierarchical action embedding method to represent the pentesting action space. It also leveraged the MITRE ATT&CK [5] to capture the relationships between actions, thereby improving the agents' performance in complex scenarios. Li et al. (2023a), Yang et al. (2025) applied the action masking method for effective exploration. Their methods dynamically eliminated invalid or low-probability actions from consideration at each state, drastically shrinking the relevant action set per step and preventing exploration of futile paths. Furthermore, incorporating expert knowledge (Chen et al., 2023) can provide domain-specific priors, which guide the agent towards promising regions of the action space and bias exploration away from irrelevant or suboptimal choices. Collectively, these approaches improved sample efficiency by focusing learning effort and enabled more effective exploration within the large action space, making learning feasible where exhaustive search or random exploration would be intractable.

Attack path planning in dynamic environments is more challenging and less explored. It necessitates agents to continuously perceive environmental shifts and rapidly adapt to evolving attack surfaces. In previous work, Yang et al. (2023), inspired by Deep Sets (Zaheer et al., 2017), proposed a flexible policy architecture named SetTron. SetTron adopted host-centric network representations that abstract individual hosts as set elements, inherently ensuring policy robustness to host ordering permutations. Li et al. (2024), on the other hand, introduced a historical information recall module and a trajectory detection module to assist the agent in monitoring scene changes, leveraging historical experience data to improve the agent's adaptability in dynamic scenarios. These methods enhanced the agent's ability to perceive and understand environmental changes, thereby successfully improving its capability to handle dynamic situations.

4.3. Limitations

Despite significant progress in RL-based attack path planning, three persistent limitations hinder the widespread applicability of these methods in real-world pentesting scenarios, particularly in black-box settings.

Limitation 1: The Simulator Fidelity Paradox. Current research (e.g., Chen et al. (2023), Li et al. (2025, 2023a, 2024, 2023b), Tran et al. (2021), Yang et al. (2025, 2023), Zhang et al. (2022), Zhou et al. (2021)) predominantly relies on network simulators to construct training environments for RL agents. While simulation enables efficient training by abstracting logical agent-environment interactions, achieving high-fidelity simulation requires complete or partial prior knowledge of the target's configuration, such as network topology, host configurations, or even exploitable vulnerabilities (as depicted in Fig. 6). This information is necessary to script realistic simulated environments capable of training agents for effective real-world deployment. Crucially, however, black-box pentesting scenarios intrinsically assume *no prior knowledge* of the target environment; such details must instead be discovered through the pentesting process itself. This creates a self-referential paradox (Łukowski, 2011): training an effective pentesting agent necessitates a high-fidelity simulator built on complete environmental knowledge, precisely the information the agent is meant to uncover in the pentesting process. Consequently, the very data required to build the training environment can only be obtained by the agent it aims to train, forming a methodological gap.

Limitation 2: Attack Paths are Scenario-Dependent. Attack path planning is inherently goal-oriented, where the target objective defines both the optimality criteria and termination conditions for the planning process. Consequently, learned penetration strategies typically operate at two decision levels (Zhou et al., 2025a): (1) *target selection* (e.g., critical hosts/assets) and (2) *action execution* against the chosen target. Crucially, the target selection policy is highly context-dependent, tightly coupled with environmental factors (e.g., network topology) and task specifications (e.g., mission-critical assets). This coupling causes learned policies to become ineffective when either the environment or task objectives change (as depicted in Fig. 7). Although prior studies (Li et al., 2024; Yang et al., 2023) have explored training agents in dynamic environments, current solutions only achieve limited adaptability under minor environmental variations.

[5] https://attack.mitre.org/

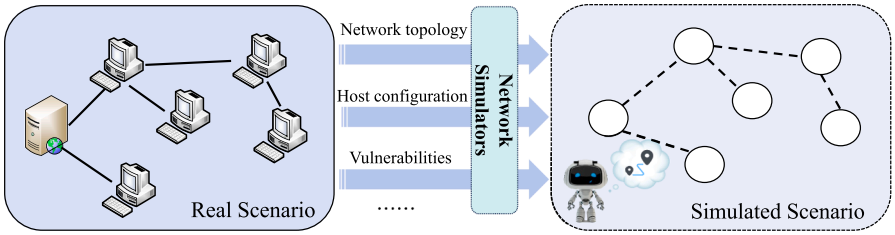

Fig. 6. Construct simulated training scenarios using Network simulators.

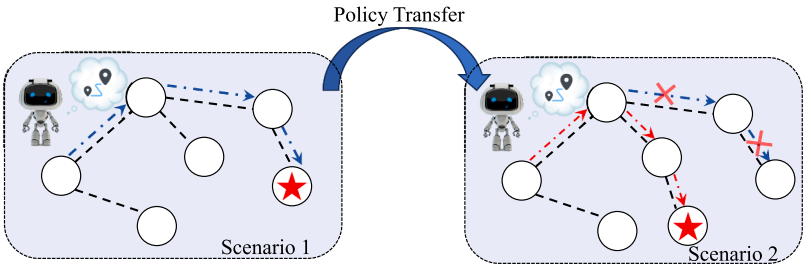

Fig. 7. Attack paths are scenario-dependent: the agent's learned attack path policy may be ineffective when transferred to another scenario.

Limitation 3: Scenario-Dependent Agent Design. In previous studies, there exist tight couplings between agents' MDP models (state/action spaces) and specific training scenarios, fundamentally constraining policy transferability across different pentesting scenarios. The reasons are threefold.

1. State space. Many approaches (e.g., Chen et al. (2023), Tran et al. (2021), Yang et al. (2025), Zhou et al. (2021)) incorporate global network attributes (e.g., topology, host count) into state spaces, causing state dimensionality to scale with environment complexity. Although some studies (e.g., Li et al. (2022, 2023a)) attempt to limit the maximum number of hosts that can be observed in the agent's state space by using fixed-size observation windows, determining the appropriate window size remains challenging and difficult to quantify.
2. State representation. State representation refers to the transformation of raw environmental observations into structured vectors consumable by neural networks. In previous studies (e.g., Chen et al. (2023), Li et al. (2022, 2023a), Tran et al. (2021), Yang et al. (2025), Zhou et al. (2021)), one-hot encoding has commonly been used. One-hot encoding is a method for converting discrete categorical variables into binary vectors. However, this approach has a significant drawback: it requires prior knowledge of the types of information to be encoded. For instance, researchers need to predefine the possible ports, services, and operating systems that might exist in the environment. If the number of categories is large, this can lead to the curse of dimensionality, where the dimensionality of the state space increases dramatically, resulting in inefficient computation and reduced model performance.
3. Action space. Some previous works (e.g., Chen et al. (2023), Tran et al. (2021), Yang et al. (2025), Zhou et al. (2021)) have typically employed abstract action types (e.g., *Exploit-HTTP* in NASim-based works) to compose the action space. However, these coarse-grained actions are difficult to translate into actual pentesting payloads in real-world scenarios. Moreover, to avoid large action spaces, they have often restricted the action space to only include actions relevant to the specific training scenario. This limitation results in an incomplete action space, thereby constraining the agent's ability to transfer policies effectively across diverse scenarios.

5. RL For penetration testing framework

In addition to attack path planning, another category of research aims to develop pentesting frameworks that can autonomously perform pentesting tasks. These studies typically do not rely on network simulators; instead, they directly target the vulnerable machines. These target machines are often vulnerability environments built using virtualization technologies, such as Vulhub [6], Vulnhub [7], and Metasploitable 2/3 [8]. Unlike simulated environments, these virtualized vulnerable machines offer greater realism in terms of functionality and interactivity, enabling the agent to interact with the environment in a manner closer to real-world conditions.

Early pentesting frameworks or tools mainly relied on rule-based approaches. They leveraged manually crafted rules or predefined workflows to select and execute corresponding payloads based on detected auxiliary information. Representative tools include classic scanning and detection tools such as Nmap [9] and Dirb [10], as well as integrated pentesting frameworks like Metasploit [11] and Cobalt Strike [12]. Rule-based frameworks or tools depend on expert knowledge to craft the rules or processes. The advantage of this approach lies in its simplicity and ease of use, and it is highly reliable in well-defined and adapted scenarios. However, it is limited by the completeness and accuracy of the rules, and typically struggles to achieve fully autonomous decision-making.

The application of RL algorithms to pentesting frameworks offers substantial potential for enhancing autonomous decision-making capabilities. For example, DeepExploit (Takaesu, 2018) employed the Asynchronous Advantage Actor-Critic (A3C) (Mnih et al., 2016) algorithm integrated with Metasploit to autonomously identify open ports on target servers and execute corresponding exploits. Zennaro and Erdődi (2023) investigated the applicability of Q-learning (Melo, 2001) in simplified Capture The Flag (CTF) environments. Besides, Maeda and Mimura (2021) specifically automated post-exploitation phases using PowerShell Empire orchestrated by the Advantage Actor-Critic (A2C) algorithm.

While these studies successfully demonstrate the feasibility of RL for pentesting decision-making, they collectively failed to address some critical limitations, like the training environment dilemma, reality gap,

[6] https://vulhub.org
[7] https://www.vulnhub.com
[8] https://docs.rapid7.com/metasploit/metasploitable-2/;https://github.com/rapid7/metasploitable3
[9] https://nmap.org/
[10] https://www.kali.org/tools/dirb/
[11] https://www.metasploit.com/
[12] https://www.cobaltstrike.com/

large action spaces, and generalization issues. To balance the training efficiency and interaction realism, Zhou et al. (2024) proposed a Real-to-Sim-to-Real pipeline to enable policy learning in realistic environments while constructing realistic simulation analogs. They also applied an LLM-powered domain randomization method to simulate the diversity of real-world environments, thereby bridging the reality gap. To enhance the agent's scalability in large action spaces, Zhou et al. (2025a) employed unstructured action descriptions as prior knowledge. This approach transformed the discrete action space into a continuous and semantically meaningful embedding space, effectively decoupling the complexity of the policy from the cardinality of the original action space. To improve the agent's generalization ability, Zhou et al. (2025b) proposed a continual RL framework to preserve the agent's past learned knowledge and resist catastrophic forgetting (Zhou & Cao, 2021) when continual learning tasks.

Existing research on RL-based pentesting frameworks is still in the early stages. Current RL-based pentesting agents still remain at a rudimentary level (Zhou et al., 2025a), capable of automating key aspects of pentesting only in simple vulnerability environments by directly invoking existing tools (such as scanning tools or exploit scripts). Designing an RL agent that can perform complex pentesting tasks in real-world environments still faces several limitations. The core limitation lies in the compromised completeness of agent state and action spaces. Current research predominantly adopts minimalist designs tailored to specific task scenarios, excluding task-agnostic information and actions. While this mitigates issues of information overload (van Veen et al., 2020) and large action spaces, which may slow learning convergence, it fundamentally constrains agents' adaptability scope. Moreover, RL agents are still severely limited in their generalization capabilities, particularly zero-shot generalization, which impedes their adaptability to novel, unseen pentesting scenarios.

6. Future perspective

Training pentesting agents to learn path planning policies via network simulators is an efficient and convenient approach. Significant progress has been made in this area, providing the community with many valuable methodologies to address key research challenges, such as sparse rewards and large action spaces. However, the idealized agent model and the absence of real environmental feedback make such agents exhibit limited adaptability in real-world black-box penetration scenarios.

Analogous to autonomous vehicles requiring end-to-end perception-decision capabilities for real-road deployment, autonomous pentesting must evolve toward integrated real-world observation, reasoning, and action execution. Currently, autonomous pentesting frameworks that do not rely on network simulators are still limited in three aspects, including agents' design, RL algorithms' poor generalization performance, and training environments. These issues severely limit agents' ability to adapt to the complex and diverse scenarios in the real world. This paper will discuss future perspectives in terms of agent design, decision algorithms, and training environments.

6.1. Agent design

Bounded and comprehensive state space. RL has achieved notable success in domains like autonomous driving (Chib & Singh, 2024) and Go game (Silver et al., 2016), where agents operate within bounded state spaces: multi-sensor fusion (e.g., LiDAR, cameras) provides comprehensive environmental perception in driving, while Go agents observe a fixed 19×19 board state. In contrast, pentesting confronts an inherently unbounded cyberspace state challenge. Reconnaissance tools (e.g., port scanners, vulnerability probes) offer only partial, fragmented visibility of network environments, lacking standardized "sensors" to acquire unified state representations essential for reliable decision-making. Consequently, agent state spaces must be designed to be bounded yet comprehensive. Such bounded-comprehensive state spaces ensure fixed-dimensional input vectors to neural networks across diverse scenarios, enabling cross-scenario policy adaptation. A currently feasible approach constrains states to the host-level information plane while exhaustively incorporating all discernible host-centric attributes (e.g., services, ports, and OS). This trade-off comes at the cost of ignoring elements like host cardinality and network topology, which are highly variable and difficult to represent with a fixed-dimensional encoding. Better alternatives still require extensive exploration.

End-to-end state representation. End-to-end training represents a critical evolution for pentesting agents, with state representations being pivotal to this paradigm. Previous methods often use one-hot encoding to represent state information as a state vector. This approach requires prior knowledge of information categories and maps them to sparse binary vectors based on these categories. However, this approach faces issues when dealing with large numbers of categories (such as 65,535 ports and services/OS types that cannot be enumerated), leading to the curse of dimensionality. Additionally, it lacks extensibility mechanisms to incorporate unseen categories during deployment. These constraints fundamentally undermine the adaptability required for true end-to-end training frameworks. In contrast, *embedding* techniques (e.g., Reimers and Gurevych (2019), Wang et al. (2021)) commonly used in natural language processing (NLP) provide a promising alternative for encoding raw state information, which is often in textual form. The textual outputs from scanning tools (e.g., service banners, website fingerprints) can be embedded into fixed-dimensional continuous vectors. Importantly, these embeddings preserve the semantic relationships within latent spaces, while naturally accommodating previously unseen categories.

Fine-grained action space. End-to-end training necessitates fine-grained action spaces, thereby directly translating agent decisions into executable commands for tool invocation and environmental interaction. Designing such action spaces can leverage the ATT&CK framework through two primary approaches: first, incorporating all Tactics, Techniques, and Procedures (TTPs) as atomic actions in the action space, which may risk a combinatorial explosion with technique proliferation; second, decomposing TTPs-based actions into macro-tactics (e.g., credential access, lateral movement) and micro-techniques (e.g., specific exploit modules), thereby applying hierarchical RL (Li et al., 2023b; Nayyar & Srivastava, 2025) for improved sample efficiency. Note that regardless of both methods, a potential challenge here is that with the updates to pentesting tools or TTPs, the action space is bound to change over time. How to maintain the effectiveness of previously learned policies in the face of evolving action spaces is an important research topic for the future.

6.2. Decision algorithm

Improving the learning efficiency and generalization ability of RL algorithms, while enabling rapid learning, policy transfer (Chen et al., 2022; Taylor & Stone, 2009), continual learning (Wang et al., 2024), and zero-shot generalization (Kirk et al., 2023) across diverse scenarios, remains a field that requires long-term exploration. While current research has proposed solutions to the key challenges outlined in Section 3.2, there is still substantial room for further study. Researchers may find inspiration by exploring related work in autonomous driving (Chen et al., 2024) and robotics (Ju et al., 2022), which face similar challenges and can offer valuable insights. Besides, exploring applications of LLMs or game-theoretic aspects in autonomous pentesting can be a promising research direction.

LLMs exhibit significant promise for intelligent decision-making in pentesting (Happe & Cito, 2025; Isozaki et al., 2025). On one hand, with their exceptional natural language understanding and context analysis capabilities, LLMs can adapt to decision-making scenarios that require processing large amounts of textual data. They can make decisions from textual information without the need for complex feature engineering

or state space definitions. On the other hand, LLMs can acquire domain knowledge via post-training and can easily generalize to unseen scenarios. Given the aforementioned advantages, the future development of an LLM-driven autonomous pentesting framework holds significant application potential.

The combination of LLMs and RL is also a promising direction for further exploration. This combination can be realized in two main paradigms. The first is a decision-making paradigm (**RL-LLM paradigm**) where the RL agent plays the primary role and the LLM serves as a supplementary tool. The second is a paradigm where the LLM takes the lead, with RL acting as a supporting mechanism (**LLM-RL paradigm**).

For the RL-LLM paradigm, the assistance of LLMs can take the form of a prior model or a guiding model to enhance the training efficiency of the RL agent in unknown environments. For instance, in Wu et al. (2025), LLMs were successfully used for action masking, improving the exploration efficiency of RL agents in large action spaces. Another feasible solution is a hierarchical decision-making architecture that combines RL and LLMs, where both components contribute at different levels to improve overall performance. For example, considering that RL algorithms offer good decision stability, their training efficiency significantly drops when the action granularity becomes too fine, resulting in a large action space. In such cases, one could use RL for coarse-grained decision-making and apply LLMs to assign parameters for pentesting tools or scripts that require customization. This would help bridge the "last mile" in autonomous pentesting frameworks.

In the LLM-RL paradigm, the focus is on leveraging RL to enhance the capabilities of LLMs. One promising research concept is to "*utilize RL agents to fine-tune LLMs via RL algorithms.*" That is, collecting high-quality trajectory data during the RL agent's training process and then using this data as training samples to fine-tune the LLM through RL algorithms like GRPO (Shao et al., 2024). On the one hand, the collected trajectories contain valuable experience data, such as environmental state information, action selections, and feedback from the environment, all of which reflect the agent's interactions. One can transform these trajectories into question-and-answer pairs, where the question corresponds to environmental state information and the answer corresponds to the action selection. On the other hand, the RL-based fine-tuning process could help the LLM to improve its ability to generate specific, context-aware responses based on the agent's experience. With this fine-tuning, the LLM would not only generate better reasoning chains but would also be more aligned with the problem-solving strategies used by the RL agent, making it more effective in tasks like pentesting.

In addition to incorporating LLMs into autonomous pentesting, future exploration could also take inspiration from recent works on game theory, such as discrete-time multiplayer games (Ge & Zhu, 2024; Zhang et al., 2017). For example, one could investigate how multi-agent RL systems can be employed, where both attacker and defender are modeled as RL agents interacting in a shared environment. This would involve designing cooperative or competitive strategies between the agents, thus enhancing the pentesting process and making it more robust against adaptive defense mechanisms. We believe that *Nash Equilibrium* (Attiah et al., 2018) or *Stackelberg games*(Speicher et al., 2018) can be applied to optimize attack-defender interactions in autonomous pentesting.

6.3. Training environment

Developing highly applicable pentesting agents for complex tasks requires the rapid generation of diverse and realistic training environments. In designing training environments, the goal is to strike a balance between environment realism and training efficiency. One feasible and promising solution to these challenges is the adoption of the digital twin technique (Masi et al., 2023) to construct emulated scenarios. Another promising approach with application potential is leveraging LLMs to automatically generate and deploy Cyber Ranges (Lupinacci et al., 2025; Yamin et al., 2024). Specifically, this can involve the use of LLMs to interpret textual descriptions of desired network environments and then automatically generate the corresponding virtual infrastructures and configurations.

7. Conclusion

This paper presents a systematic review of RL-based autonomous pentesting and provides future perspectives. In this work, we highlight the key challenges associated with applying RL in autonomous pentesting, offering an in-depth analysis of the latest developments and limitations in RL-based attack path planning and autonomous pentesting frameworks. To the best of our knowledge, this is the first systematic review in this area. While significant progress has been made, we recognize that much work remains before autonomous pentesting can be effectively deployed in the complex and dynamic scenarios of the real world. Many research challenges still need to be addressed, which presents a compelling opportunity for continued investigation and innovation in this field.

CRediT authorship contribution statement

Jingju Liu: Writing – Original Draft, Conceptualization, Formal analysis, Investigation; **Yue Zhang:** Writing – Original Draft, Conceptualization, Formal analysis, Investigation; **Shicheng Zhou:** Writing – Review & Editing, Visualization, Conceptualization; **Jiahai Yang:** Supervision, Writing – Review & Editing; **Yuliang Lu:** Supervision, Project administration, Writing – Review & Editing; **Xiaofeng Zhong:** Writing – Review & Editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Abu-Dabaseh, F., & Alshammari, E. (2018). Automated penetration testing : an overview. In *Computer science & information technology* (pp. 121–129). Academy & Industry Research Collaboration Center (AIRCC). https://doi.org/10.5121/csit.2018.80610

Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, *34*(6), 26–38. https://doi.org/10.1109/MSP.2017.2743240

Attiah, A., Chatterjee, M., & Zou, C. C. (2018). A game theoretic approach to model cyber attack and defense strategies. In *2018 IEEE International conference on communications, ICC 2018, kansas city, mo, usa, may 20–24, 2018* (pp. 1–7). IEEE. https://doi.org/10.1109/ICC.2018.8422719

Baillie, C., Standen, M., Schwartz, J., Docking, M., Bowman, D., & Kim, J. (2020). CybORG: An autonomous cyber operations research gym. *CoRR*, *abs/2002.10667*.

Brehmer, B. (2005). The dynamic OODA loop: Amalgamating boyd's OODA loop and the cybernetic approach to command and control. *Proceedings International Command &Control Research & Technology Symposium*, .

Chen, J., Hu, S., Zheng, H., Xing, C., & Zhang, G. (2023). Gail-pt: An intelligent penetration testing framework with generative adversarial imitation learning. *Computers & Security*, *126*, 103055.

Chen, L., Wu, P., Chitta, K., Jaeger, B., Geiger, A., & Li, H. (2024). End-to-end autonomous driving: Challenges and frontiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*(12), 10164–10183. https://doi.org/10.1109/TPAMI.2024.3435937

Chen, X., Hu, J., Jin, C., Li, L., & Wang, L. (2022). Understanding domain randomization for sim-to-real transfer. In *The tenth international conference on learning representations, ICLR 2022, virtual event, april 25–29, 2022* (pp. 1–28). OpenReview.net.

Chen, Y., Chen, Y., Yang, Y., Li, Y., Yin, J., & Fan, C. (2019). Learning action-transferable policy with action embedding. *CoRR*, *abs/1909.02291*.

Chen, Y., He, J., Fang, W., Yang, S., Chen, J., Li, T., & Lan, X. (2025). Automatic penetration testing model based on reinforcement learning for complex network environments. *The Journal of supercomputing*, *81*(11), 1169. https://doi.org/10.1007/s11227-025-07656-2

Chib, P. S., & Singh, P. (2024). Recent advancements in end-to-end autonomous driving using deep learning: A survey. *IEEE Transactions on Intelligent Vehicles*, *9*(1), 103–118. https://doi.org/10.1109/TIV.2023.3318070

Cobbe, K., Klimov, O., Hesse, C., Kim, T., & Schulman, J. (2019). Quantifying generalization in reinforcement learning. In K. Chaudhuri, & R. Salakhutdinov (Eds.), *Proceedings of the 36th international conference on machine learning, ICML 2019, 9–15 june 2019, long beach, california, USA* (pp. 1282–1289). PMLR (*vol. 97*). Proceedings of Machine Learning Research.

Deng, G., Liu, Y., Vilches, V. M., Liu, P., Li, Y., Xu, Y., Pinzger, M., Rass, S., Zhang, T., & Liu, Y. (2024). PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In D. Balzarotti, & W. Xu (Eds.), *33Rd USENIX security symposium, USENIX security 2024, philadelphia, pa, usa, august 14–16, 2024*. USENIX Association.

Dulac-Arnold, G., Evans, R., Hasselt, H. V., Sunehag, P., Lillicrap, T. P., Hunt, J. J., Mann, T. A., Weber, T., Degris, T., & Coppin, B. (2015). Deep reinforcement learning in large discrete action spaces. arXiv: Artificial Intelligence.

Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., & Liu, H. X. (2023). Dense reinforcement learning for safety validation of autonomous vehicles. *Nature*, *615*, 620–627.

Fourati, F., Aggarwal, V., & Alouini, M. (2024). Stochastic q-learning for large discrete action spaces. In *Forty-first international conference on machine learning, ICML 2024, vienna, austria, july 21–27, 2024* (pp. 1–30). OpenReview.net.

François-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G., & Pineau, J. (2018). An Introduction to Deep Reinforcement Learning. Now Foundations and Trends. https://doi.org/10.1561/2200000071

Ge, Y., & Zhu, Q. (2024). MEGA-PT: A meta-game framework for agile penetration testing. In A. Sinha, J. Fu, Q. Zhu, & T. Zhang (Eds.), *Decision and game theory for security - 15th international conference, gamesec 2024, new york city, NY, USA, october 16–18, 2024, proceedings* (pp. 24–44). Springer (*vol. 14908*). Lecture Notes in Computer Science. https://doi.org/10.1007/978-3-031-74835-6_2

Happe, A., & Cito, J. (2025). Benchmarking practices in LLM-driven offensive security: Testbeds, metrics, and experiment design. *CoRR*, *abs/2504.10112*. https://doi.org/10.48550/ARXIV.2504.10112

Hoffmann, J. (2015). Simulated penetration testing: From "dijkstra" to "turing test++". In *Twenty-fifth international conference on automated planning and scheduling*.

Holm, H. (2023). Lore a red team emulation tool. *IEEE Transactions on Dependable and Secure Computing*, *20*(2), 1596–1608. https://doi.org/10.1109/TDSC.2022.3160792

Hu, Z., Beuran, R., & Tan, Y. (2020). Automated penetration testing using deep reinforcement learning. *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, (pp. 2–10).

Isozaki, I., Shrestha, M., Console, R., & Kim, E. (2025). Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements. In *Adjunct proceedings of the 33rd ACM conference on user modeling, adaptation and personalization, UMAP adjunct 2025, new york city, ny, usa, june 16–19, 2025* (pp. 404–419). ACM. https://doi.org/10.1145/3708319.3733804

Ju, H., Juan, R., Gomez, R., Nakamura, K., & Li, G. (2022). Transferring policy of deep reinforcement learning from simulation to reality for robotics. *Nature Machine Intelligence*, *4*(12), 1077–1087. https://doi.org/10.1038/S42256-022-00573-6

Kirk, R., Zhang, A., Grefenstette, E., & Rocktäschel, T. (2023). A survey of zero-shot generalisation in deep reinforcement learning. *The Journal of Artificial Intelligence Research*, *76*, 201–264. https://doi.org/10.1613/JAIR.1.14174

Kobayashi, M., Fuchi, M., Zanashir, A., Yoneda, T., & Takagi, T. (2025). Construction and evaluation of LLM-based agents for semi-autonomous penetration testing.

Li, L., El Rami, J.-P. S., Taylor, A., Rao, J. H., & Kunz, T. (2022). Enabling a network AI gym for autonomous cyber agents. In *2022 International conference on computational science and computational intelligence (CSCI)* (pp. 172–177). https://doi.org/10.1109/CSCI58124.2022.00034

Li, M., Zhu, T., Yan, H., Chen, T., & Lv, M. (2025). HER-PT: An intelligent penetration testing framework with hindsight experience replay. *Computers & Security*, *152*, 104357. https://doi.org/10.1016/J.COSE.2025.104357

Li, Q., Hu, M., Hao, H., Zhang, M., & Li, Y. (2023a). INNES: An intelligent network penetration testing model based on deep reinforcement learning. *Applied Intelligence*, *53*(22), 27110–27127. https://doi.org/10.1007/S10489-023-04946-1

Li, Q., Wang, R., Li, D., Shi, F., Zhang, M., Chattopadhyay, A., Shen, Y., & Li, Y. (2024). Dynpen: Automated penetration testing in dynamic network scenarios using deep reinforcement learning. *IEEE Transactions on Information Forensics and Security*, *19*, 8966–8981. https://doi.org/10.1109/TIFS.2024.3461950

Li, Q., Zhang, M., Shen, Y., Wang, R., Hu, M., Li, Y., & Hao, H. (2023b). A hierarchical deep reinforcement learning model with expert prior knowledge for intelligent penetration testing. *Computers & Security*, *132*, 103358. https://doi.org/10.1016/J.COSE.2023.103358

Łukowski, P. (2011). Paradoxes of Self-Reference. Dordrecht: Springer Netherlands. https://doi.org/10.1007/978-94-007-1476-2_4

Lupinacci, M., Blefari, F., Romeo, F., Pironti, F. A., & Furfaro, A. (2025). Arcer: An agentic RAG for the automated definition of cyber ranges. *CoRR*, *abs/2504.12143*. https://doi.org/10.48550/ARXIV.2504.12143

Maeda, R., & Mimura, M. (2021). Automating post-exploitation with deep reinforcement learning. *Computers & Security*, *100*, 102108.

Masi, M., Sellitto, G. P., Aranha, H., & Pavleska, T. (2023). Securing critical infrastructures with a cybersecurity digital twin. *Software and Systems Modeling*, *22*(2), 689–707. https://doi.org/10.1007/S10270-022-01075-0

Melo, F. S. (2001). Convergence of q-learning: A simple proof. *Institute Of Systems and Robotics, Tech. Rep*, (pp. 1–4).

Midian, P. (2002). Perspectives on penetration testing — black box vs. white box. *Network Security*, *2002*(11), 10–12. https://doi.org/10.1016/S1353-4858(02)11009-9

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In M. Balcan, & K. Q. Weinberger (Eds.), *Proceedings of the 33nd international conference on machine learning, ICML 2016, new york city, ny, usa, june 19–24, 2016* (pp. 1928–1937). JMLR.org (*vol. 48*). JMLR Workshop and Conference Proceedings.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533.

Moskal, S., Laney, S., Hemberg, E., & O'Reilly, U. (2023). Llms killed the script kiddie: How agents supported by large language models change the landscape of network threat testing. *CoRR*, arXiv:2310.06936. https://doi.org/10.48550/ARXIV.2310.06936

Nayyar, R. K., & Srivastava, S. (2025). Autonomous option invention for continual hierarchical reinforcement learning and planning. In T. Walsh, J. Shah, & Z. Kolter (Eds.), *Aaai-25, sponsored by the association for the advancement of artificial intelligence, february 25, - march 4, 2025, philadelphia, pa, USA* (pp. 19642–19650). AAAI Press. https://doi.org/10.1609/AAAI.V39I18.34163

Nguyen, H. P. T., Hasegawa, K., Fukushima, K., & Beuran, R. (2025). Pengym: Realistic training environment for reinforcement learning pentesting agents. *Computers & Security* , *148*, 104140. https://doi.org/10.1016/J.COSE.2024.104140

Nguyen, H. V., & Uehara, T. (2022). Hierarchical action embedding for effective autonomous penetration testing. In *2022 IEEE 22Nd international conference on software quality, reliability, and security companion (QRS-c)* (pp. 152–157). https://doi.org/10.1109/QRS-C57518.2022.00030

Obes, J. L., Sarraute, C., Richarte, G. (2013). Attack planning in the real world. In *Secart*.

Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning* (pp. 2778–2787). PMLR.

Phillips, C., & Swiler, L. P. (1998). A graph-based system for network-vulnerability analysis. In *Proceedings of the 1998 workshop on new security paradigms* (pp. 71–79).

Rane, N., & Qureshi, A. (2024). Comparative analysis of automated scanning and manual penetration testing for enhanced cybersecurity. In *12Th international symposium on digital forensics and security, ISDFS 2024, san antonio, tx, usa, april 29–30, 2024* (pp. 1–6). IEEE. https://doi.org/10.1109/ISDFS60797.2024.10527240

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In K. Inui, J. Jiang, V. Ng, & X. Wan (Eds.), *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing, EMNLP-IJCNLP 2019, hong kong, china, november 3–7, 2019* (pp. 3980–3990). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Ren, Q., Liu, J., Xiong, X., & Lu, C. (2024a). Automated penetration testing based on LSTM and advanced curiosity exploration. In *2024 IEEE 5Th international conference on pattern recognition and machine learning (PRML)* (pp. 150–156). https://doi.org/10.1109/PRML62565.2024.10779693

Ren, Q., Xiong, X., & Liu, J. (2024b). E2e-Autopt: An end-to-end automated penetration testing with LSTM-PPO approach. In *Network and parallel computing - 20th IFIP WG 10.3 international conference, NPC 2024, haikou, china, december 7–8, 2024, proceedings, part II* (pp. 403–416). Springer (*vol. 15528*). Lecture Notes in Computer Science. https://doi.org/10.1007/978-981-96-2864-3_32

Saber, V., ElSayad, D., Bahaa-Eldin, A. M., & Fayed, Z. (2023). Automated penetration testing, a systematic review. In *2023 International mobile, intelligent, and ubiquitous computing conference (MIUCC)* (pp. 373–380). https://doi.org/10.1109/MIUCC58832.2023.10278377

Sarraute, C. (2013). Automated attack planning. *CoRR*, *abs/1307.7808*.

Sarraute, C., Buffet, O., & Hoffmann, J. (2012). Pomdps make better hackers: Accounting for uncertainty in penetration testing. In *Twenty-sixth AAAI conference on artificial intelligence*.

Sarraute, C., Buffet, O., & Hoffmann, J. (2013). Penetration testing== pomdp solving? arXiv:1306.4714

Schneier, B. (1999). Attack trees. *Dr. Dobb'is journal*, *24*(12), 21–29.

Schwartz, J., Kurniawati, H., & El-Mahassni, E. (2020). Pomdp+ information-decay: Incorporating defender's behaviour in autonomous penetration testing. In *Proceedings of the international conference on automated planning and scheduling* (pp. 235–243). (*vol. 30*).

Schwartz, J., & Kurniawatti, H. (2019). Nasim: Network attack simulator. https://github.com/Jjschwartz/NetworkAttackSimulator.

Seifert, C., Betser, M., Blum, W., Bono, J., Farris, K., Goren, E., Grana, J., Holsheimer, K., Marken, B., Neil, J., Nichols, N., Parikh, J., & Wei, H. (2021). Cyberbattlesim. https://github.com/microsoft/cyberbattlesim. Created by Microsoft Defender Research Team.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, arXiv:2402.03300. https://doi.org/10.48550/ARXIV.2402.03300

Sheyner, O., Haines, J., Jha, S., Lippmann, R., & Wing, J. M. (2002). Automated generation and analysis of attack graphs. In *Proceedings 2002 IEEE symposium on security and privacy* (pp. 273–284). IEEE.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016). Mastering the game of go with deep neural networks and tree search. *Nature*, *529*(7587), 484–489.

Silver, D., Singh, S., Precup, D., & Sutton, R. S. (2021). Reward is enough. *Artificial Intelligence*, (p. 103535).

Simon, R., & Mees, W. (2024). Sok: A comparison of autonomous penetration testing agents. In *Proceedings of the 19th international conference on availability, reliability and security, ARES 2024, vienna, austria, 30 july 2024 - 2 august 2024* (pp. 41:1–41:10). ACM. https://doi.org/10.1145/3664476.3664484

Skandylas, C., & Asplund, M. (2025). Automated penetration testing: Formalization and realization. *Computers & Security* , *155*, 104454. https://doi.org/10.1016/J.COSE.2025.104454

Speicher, P., Steinmetz, M., Backes, M., Hoffmann, J., & Künnemann, R. (2018). Stackelberg planning: Towards effective leader-follower state space search. In S. A. McIlraith, & K. Q. Weinberger (Eds.), *Proceedings of the thirty-second AAAI conference on artificial intelligence, (aaai-18), the 30th innovative applications of artificial intelligence (iaai-18), and the 8th AAAI symposium on educational advances in artificial intelligence*

(eaai-18), new orleans, louisiana, usa, february 2–7, 2018 (pp. 6286–6293). AAAI Press. https://doi.org/10.1609/AAAI.V32I1.12090

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT press.

Takaesu, I. (2018). Deepexploit. https://github.com/13o-bbr-bbq/machine_learning_security/blob/master/DeepExploit.

Taylor, M. E., & Stone, P. (2009). Transfer learning for reinforcement learning domains: a survey. *Journal of Machine Learning Research : JMLR*, *10*, 1633–1685. https://doi.org/10.5555/1577069.1755839

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International conference on intelligent robots and systems, IROS 2017, vancouver, bc, canada, september 24–28, 2017* (pp. 23–30). IEEE. https://doi.org/10.1109/IROS.2017.8202133

Tran, K., Akella, A., Standen, M., Kim, J., Bowman, D., Richer, T., & Lin, C.-T. (2021). Deep hierarchical reinforcement agents for automated penetration testing. arXiv e-prints, arXiv:2109.06449). https://doi.org/10.48550/arXiv.2109.06449

van Veen, D.-J., Kudesia, R. S., & Heinimann, H. R. (2020). An agent-based model of collective decision-making: how information sharing strategies scale with information overload. *IEEE Transactions on Computational Social Systems*, *7*(3), 751–767. https://doi.org/10.1109/TCSS.2020.2986161

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T. P., Kavukcuoglu, K., Hassabis, D., Apps, C., & Silver, D. (2019). Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, (pp. 1–5).

Wang, K., Kang, B., Shao, J., & Feng, J. (2020). Improving generalization in reinforcement learning with mixture regularization. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems 33: Annual conference on neural information processing systems 2020, neurIPS 2020, december 6–12, 2020, virtual*.

Wang, K., Reimers, N., & Gurevych, I. (2021). Tsdae: Using transformer-based sequential denoising auto-encoderfor unsupervised sentence embedding learning. arXiv:2104.06979

Wang, L., Zhang, X., Su, H., & Zhu, J. (2024). A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *PP*. https://doi.org/10.1109/tpami.2024.3367329

Wang, Z., Chen, C., & Dong, D. (2023). A dirichlet process mixture of robust task models for scalable lifelong reinforcement learning. *IEEE Transactions on Cybernetics*, *53*(12), 7509–7520. https://doi.org/10.1109/TCYB.2022.3170485

Weiyi, Y., Chenjia, B., Chao, C. et al. (2020). Survey on sparse reward in deep reinforcement learning. *Journal of Computer Science*, *47*.

Wenzel, P., Yang, N., Wang, R., Zeller, N., & Cremers, D. (2025). 4Seasons: Benchmarking visual SLAM and long-term localization for autonomous driving in challenging conditions. *International Journal of Computer Vision*, *133*(4), 1564–1586. https://doi.org/10.1007/S11263-024-02230-4

Wu, L., Liu, J., Yang, J., & Zhong, X. (2025). Automated penetration testing through hierarchical PPO with large language model enhancement. In D. Huang, H. Chen, B. Li, & Q. Zhang (Eds.), *Advanced intelligent computing technology and applications - 21st international conference, ICIC 2025, ningbo, china, july 26–29, 2025, proceedings, part XIV* (pp. 472–486). Springer (*vol. 15855*). Lecture Notes in Computer Science. https://doi.org/10.1007/978-981-96-9908-7_39

Wu, L., Zhong, X., Liu, J., & Wang, X. (2024). PTGroup: An automated penetration testing framework using llms and multiple prompt chains. In *International conference on intelligent computing* (pp. 220–232). Singapore: Springer Nature Singapore.

Xu, Z., Tang, C., & Tomizuka, M. (2018). Zero-shot deep reinforcement learning driving policy transfer for autonomous vehicles based on robust control. In W. Zhang, A. M. Bayen, J. J. S. Medina, & M. J. Barth (Eds.), *21St international conference on intelligent transportation systems, ITSC 2018, maui, hi, usa, november 4–7, 2018* (pp. 2865–2871). IEEE. https://doi.org/10.1109/ITSC.2018.8569612

Yadav, T., & Rao, A. M. (2015). Technical aspects of cyber kill chain. In *International symposium on security in computing and communication* (pp. 438–452). Springer.

Yamin, M. M., Hashmi, E., Ullah, M., & Katt, B. (2024). Applications of LLMs for generating cyber security exercise scenarios. *IEEE Access*, *12*, 143806–143822. https://doi.org/10.1109/ACCESS.2024.3468914

Yang, Y., Chen, L., Liu, S., Wang, L., Fu, H., Liu, X., & Chen, Z. (2025). Behaviour-diverse automatic penetration testing: A coverage-based deep reinforcement learning approach. *Frontiers of Computer Science*, *19*(3), 193309. https://doi.org/10.1007/S11704-024-3380-1

Yang, Y., Chen, M., Fu, H., & Liu, X. (2023). Settron: Towards better generalisation in penetration testing with reinforcement learning. In *IEEE Global communications conference, GLOBECOM 2023, kuala lumpur, malaysia, december 4–8, 2023* (pp. 4662–4667). https://doi.org/10.1109/GLOBECOM54140.2023.10437804

Yousefi, M., Mtetwa, N., Zhang, Y., & Tianfield, H. (2018). A reinforcement learning approach for attack graph analysis. In *17Th IEEE international conference on trust, security and privacy in computing and communications / 12th IEEE international conference on big data science and engineering, trustcom/bigdatase 2018, new york, ny, usa, august 1–3, 2018* (pp. 212–217). IEEE. https://doi.org/10.1109/TRUSTCOM/BIGDATASE.2018.00041

Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., & Smola, A. J. (2017). Deep sets. In I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems 30: annual conference on neural information processing systems 2017, december 4–9, 2017, long beach, ca, USA* (pp. 3391–3401).

Zennaro, F. M., & Erdődi, L. (2023). Modelling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs between model-free learning and a priori knowledge. *IET Information Security*, *17*(3), 441-457. https://doi.org/10.1049/ise2.12107

Zhang, H., Jiang, H., Luo, C., & Xiao, G. (2017). Discrete-time nonzero-sum games for multiplayer using policy-iteration-based adaptive dynamic programming algorithms. *IEEE Transactions on Cybernetics*, *47*(10), 3331–3340. https://doi.org/10.1109/TCYB.2016.2611613

Zhang, Y., Liu, J., Zhou, S., Hou, D., Zhong, X., & Lu, C. (2022). Improved deep recurrent q-network of POMDPs for automated penetration testing. *Applied Sciences*, *12*(20). https://www.mdpi.com/2076-3417/12/20/10339. https://doi.org/10.3390/app122010339

Zhao, W., Queralta, J. P., & Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: A survey. In *2020 IEEE Symposium series on computational intelligence, SSCI 2020, canberra, australia, december 1–4, 2020* (pp. 737–744). IEEE. https://doi.org/10.1109/SSCI47803.2020.9308468

Zhou, F., & Cao, C. (2021). Overcoming catastrophic forgetting in graph neural networks with experience replay. In *Thirty-fifth AAAI conference on artificial intelligence, AAAI 2021, thirty-third conference on innovative applications of artificial intelligence, IAAI 2021, the eleventh symposium on educational advances in artificial intelligence, EAAI 2021, virtual event, february 2–9, 2021* (pp. 4714–4722). https://doi.org/10.1609/AAAI.V35I5.16602

Zhou, S., Liu, J., Hou, D., Zhong, X., & Zhang, Y. (2021). Autonomous penetration testing based on improved deep q-network. *Applied Sciences*, *11*(19). https://doi.org/10.3390/app11198823

Zhou, S., Liu, J., Lu, Y., Yang, J., Hou, D., Zhang, Y., & Hu, S. (2025a). April: Towards scalable and transferable autonomous penetration testing in large action space via action embedding. *IEEE Transactions on Dependable and Secure Computing*, *22*(3), 2443–2459. https://doi.org/10.1109/TDSC.2024.3518500

Zhou, S., Liu, J., Lu, Y., Yang, J., Zhang, Y., & Chen, J. (2024). Towards generalizable autonomous penetration testing via domain randomization and meta-Reinforcement learning. arXiv e-prints, arXiv:2412.04078. https://doi.org/10.48550/arXiv.2412.04078

Zhou, S., Liu, J., Lu, Y., Yang, J., Zhang, Y., Lin, B., Zhong, X., & Hu, S. (2025b). SCRIPT: A scalable continual reinforcement learning framework for autonomous penetration testing. *Expert Systems with Applications*, *285*, 127827. https://doi.org/10.1016/J.ESWA.2025.127827

Zhou, T.-y., Zang, Y.-c., Zhu, J.-h., & Wang, Q.-x. (2019). Nig-ap: A new method for automated penetration testing. *Frontiers of Information Technology & Electronic Engineering*, *20*(9), 1277–1288.